An Operating System Simulator

COMP2007

Operating Systems and Concurrency Coursework
Specification v4.00

Dan Marsden and Chao Chen

2026

Contents

1 Introduction **4**

2 The simulator design **4**

3 Getting started **6**

3.1 Working on the department Linux environment 6

3.2 Extracting the files 6

3.3 What's provided? 6

3.4 Using the code 7

4 Project instructions **8**

4.1 Implementing the logger 10

4.2 Implementing a non-blocking queue 10

4.3 Implementing a bounded queue 12

4.4 Implementing priority queues 14

4.5 Simulating multiple CPUs 15

4.6 Managing process ids 15

4.7 Simulating running processes and waiting for them to finish . . . 16

4.8 Making the environment create and wait for processes 17

4.9 Dealing with blocking processes 18

4.10 Making the environment create and wait for jobs that may block 18

4.11 Priority Boosting 19

4.12 CPU Load Rebalancing 19

5 Submitting your work **20**

6 Where to find help **21**

7 Assessment guidelines **21**

7.1 General guidelines 21

7.2 AI Usage 22

8 Academic Misconduct **22**

Appendices **23**

A Background on assertions **23**

B Using the evaluator **24**

1 Introduction

The purpose of this year's coursework is to implement a simulator of an operating system. Completing this coursework will require understanding of both operating system design and multi-threaded programming. Once the concurrency topic has been covered in lectures, you should have all the background that you need to complete this coursework.

Several simplifying assumptions are made to keep the task manageable. In particular:

- Requests for the operating system will come from an "external simulated environment" rather than from other processes managed by the operating system of the machine you are working on.
- IO will occur periodically rather than in response to real IO requests.
- We only deal with process management. There will be no threading, memory, or file management component in the simulator.
- CPU activity will be simulated in a simple manner provided as part of the starting code on Moodle.

2 The simulator design

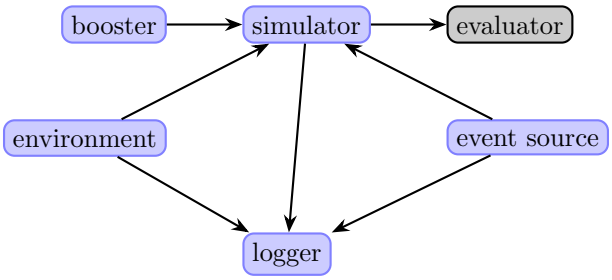


Figure 1: Core components

The core components of the simulator are shown in Figure 1 (the blue components require modification or implementation in your project). Their purposes is explained in more detail below.

The simulator pretends to be a simple operating systems. It provides:

- functions to start and stop it running.
- functions to simulate creating new processes, wait for processes to terminate, and to boost process priority.
- a function to receive notifications of IO events.
- the necessary data structures to manage processes and run them on the evaluator.

The evaluator pretends to run processes. It provides:

- a function to run some code from a chosen program counter value.
- various "programs" that can be run by the evaluator as part of the simulation.

The environment pretends to be the "outside world" requesting functionality of the operating system. It provides:

- functions to start and stop it running.
- machinery to simulate requests to create and wait for the termination of various processes, via requests to the simulator.

The event source notifies the simulator of IO events. It provides:

- functions to start and stop it running.
- machinery to simulate IO events, via calls to the simulator.

The priority booster implements dynamic process priorities. It provides:

- functions to start and stop it running.
- machinery to simulate priority boosting, via calls to the simulator.

The logger implements logging functionality for the various other components. It provides:

- functions to start and stop it running.
- a function to write numbered log, debug, and error messages to the terminal.

The source code available on Moodle includes a complete implementation of the evaluator and trivial implementations of the interface functions required of the other components. Filling in the remaining functionality will be the aim of the coursework.

This initial description will probably be hard to digest, and may be overwhelming. The sections below provide more details, and break the task down into more manageable parts.

3 Getting started

This section will describe how to setup the initial source code for your coursework, what it comprises, and some first steps in working with it.

3.1 Working on the department Linux environment

Details for logging into the School Linux environment, and how to copy files onto the Linux servers can be found in Moodle.

3.2 Extracting the files

To use the code provided for the project, complete the following steps:

1. Copy `coursework.tar.gz` to the directory where you intend to work. It is probably best to make a fresh sub-directory for this purpose, as when you extract the files, they will all appear directly in this directory.
2. Check the permissions set on the directory. You must change them to give read, write and execute permissions to the owner, and revoke all rights from the user group and rest of the world. You can achieve this using `chmod u=rwx,go= <directory name>`. Note the space before the directory name.
3. Change directory to the directory you copied `coursework.tar.gz` into.
4. Extract the zip file using the command `tar zxvf coursework.tar.gz`.

The directory now contains the zip file, several C header and source files, and a makefile. These are described in more detail in Section 3.3.

3.3 What's provided?

If you have followed the instructions in Section 3.2 your directory contains the following:

- A makefile `Makefile` which will be used to build your project, and build and run tests, and to package up your final source code for submission.

The makefile already has the features needed to support the entire project. Do not worry if you haven't seen makefiles before. You will be told exactly which commands you need to run to complete various tasks, and no knowledge of the make tool is required.

- Various C header (.h) and source (.c) files. These are both the files you will add code to during the course of the project, and some machinery that is provided for you.

Some of the C code provides functionality to help with the project:

- `list.h` and `list.c` are an implementation of a doubly linked list of `unsigned int` values.
- `list.tests.c` contains unit tests for the linked list implementation.
- `evaluator.h` and `evaluator.c` are an implementation of the evaluator component.
- `evaluator.tests.c` contains unit tests for the evaluator component.
- `coursework.c` combines the various components together to form a complete application. You will not need to modify this file during your work, as how the components interact will remain fixed.

Important: Please do not modify the source files or makefile listed above. If you do so accidentally, just replace them with a fresh copy taken from the provided zip file. **The makefile will be used as part of the assessment of your code. Working with a modified makefile may endanger your marks.**

3.4 Using the code

As a first step, you can compile the coursework using the command:

```
make coursework
```

This should run a series of compilation steps, and complete successfully. Once this has completed, an executable file named `coursework` appears in the same directory. You can run this executable, e.g., as follows:

```
./coursework true false false
```

The program should quickly complete successfully, but without printing anything out, or doing anything interesting. It will do more as you start adding your own code following the steps outlined in Section 4.

There are also some other things you can do with the provided makefile. Type the command:

```
make list.tested
```

It will compile and then run some unit tests for the provided linked list implementation. The tests should succeed, leaving a new file called `list.tested`.

Similarly, you can run some unit tests for the evaluator by typing:

```
make evaluator.tested
```

Again, this should compile and run some tests successfully, leaving a new file `evaluator.tested` if it did so.

Finally, type:

```
make clean
```

It will remove all the files generated by the makefile. It can be useful to clean if your build gets into a state you don't understand, or if you just want to prepare to run the whole build from scratch. Note this will even remove the zip file, as this can be regenerated from the source files by typing:

```
make coursework.tar.gz
```

This feature will be used in Section 5, which describes how to submit your work.

4 Project instructions

The following general instructions apply to the various coding tasks needed to complete the project.

- Please don't modify the files you were asked to leave unchanged in Section 3.3.
- Please don't add new source files. The coursework is designed so you only need to modify some of the provided files, and the marking process will assume this is how things have been done.

- You may use the C standard library, the standard Unix libraries, and the Posix threading library (pthreads). Please do not introduce other libraries to the project.
- You should avoid busy waiting in your solution wherever possible.
- For the other provided (.c) and (.h) files:
  - Some function definitions are already provided, and you will be asked to fill in the details in the various parts below.
  - You can add other code such as data types, global variables, and other functions in these files as needed to complete the tasks.
  - The files `utilities.h` and `utilities.c` can be used for code that you want to reuse in different parts of the project. Place new declarations in the header (.h) file, and the corresponding definitions in the source (.c) file. To get you started, we have provided two utility functions:
    * `checked_malloc` - A wrapper around the usual malloc function which aborts if the memory allocation fails. The coursework does not require you to handle memory allocation failures, but knowing they have occurred may help reveal other bugs.
    * `checked_free` - A wrapper around the usual free function that asserts it hasn't been passed a null pointer.

    You can use these functions if they prove useful in your projects. You should also follow the same pattern if you introduce other utility functions of your own.
- To provide a double-check to make sure student work is correctly associated with the right student, if you modify a C source file (.c file), please add a comment on the top line with your name and student ID in. For example:

```
// Student: Ada Lovelace ID: 123456789
```

- You will be asked to add log messages at various points in the code. The precise format does not matter, as long as at least the requested information appears in the message.
- We strongly recommend version controlling your work using the provided git repository.

Before starting, it might be worth reading the assessment guidance in Section 7 to understand how your work will be evaluated.

Important: It is also recommended that you read all the tasks required to implement the coursework first, as later steps may impact earlier design decisions.

4.1 Implementing the logger

In this step, the aim is to implement a logging facility that can be used by various parts of the simulator, potentially from multiple threads. To do so you should provide an implementation of the `logger_write`, `debug_write`, and `priority_write`, functions in `logger.c`. These functions should:

- Write out each log/debug/priority message to standard output (the terminal) with a unique message number which rises in increments of one, starting at zero.
- Write out the time associated with each message.
- Write out each message on a separate line.

It should be possible to run your code with different levels of "debugging" by enabling/disabling different streams on the command line. Sample output might be:

```
52008 : 09:31:55 : [Log] Simulator thread 0 terminated
52011 : 09:31:55 : [Log] Stopping simulator
```

Note that the time may not be accurate enough to increase every message, but the message counter will. You may need to add some setup and shutdown code to the logger. If so, modify the implementations of `logger_start` and `logger_stop` accordingly.

To check your implementation, you can recompile and run `coursework`. You should now see a couple of messages printed to the terminal during a run of the program. The three separate logger streams (log, debug, and priority) should be used appropriately in the remainder of this project, i.e.:

- Log functionality to report progress on the simulation, such as starting and stopping components, creating and running processes, and so on.
- Debug for detailed diagnostics.
- Priority for error messages enabling posterior analysis in "production".

4.2 Implementing a non-blocking queue

The aim of this section is to implement a non-blocking queue of `unsigned int` values which may be useful in later parts. A **non-blocking queue** is a queue

where pop calls fail if no data is available, rather than blocking until another thread pushes a value.

If you don't require this component later in your coursework, **you should still implement it** as the code will be assessed. You should use the provided doubly linked list implementation to help you in this task.

To get you started, an outline implementation is provided in `non_blocking_queue.h` and `non_blocking_queue.c`. The header file has the following contents:

```
#ifndef _NON_BLOCKING_QUEUE_H_
#define _NON_BLOCKING_QUEUE_H_

#include "list.h"

typedef struct NonBlockingQueue {
  /* Add fields as needed */
  int remove_me; // Added to silence empty structure compiler warning
} NonBlockingQueueT;

void non_blocking_queue_create(NonBlockingQueueT* queue);
void non_blocking_queue_destroy(NonBlockingQueueT* queue);

void non_blocking_queue_push(NonBlockingQueueT* queue, unsigned int value);
int non_blocking_queue_pop(NonBlockingQueueT* queue, unsigned int* value);

int non_blocking_queue_empty(NonBlockingQueueT* queue);
unsigned int non_blocking_queue_length(NonBlockingQueueT* queue);

#endif
```

You should modify the code in `non_blocking_queue.c` as follows:

- Add code so `non_blocking_queue_create` and `non_blocking_queue_destroy` create and destroy a queue, allocating and releasing resources as required. To do so, you may need to add fields to the definition of `NonBlockingQueue` in `non_blocking_queue.h`.
- Add code so that `non_blocking_queue_empty` returns non-zero if and only if the queue is empty.
- Add code so that `non_blocking_queue_length` returns the length of the queue.
- Add code so that `non_blocking_queue_push` adds the value to the end of the queue.
- Add code to `non_blocking_queue_pop` so that:
  - if the queue is non-empty, it removes the front of the queue, stores it in `*value`, and returns zero.
  - otherwise, it returns a non-zero value indicating failure. That is, pop **fails rather than blocking** waiting for a value to be pushed.

You may also need to add other supporting code in these files, such as suitable type definitions or additional functions. You can also add or use functions from the files `utilities.h` and `utilities.c`.

The queue should be designed so it is safe to use in multi-threaded code.

You also should write some tests for your code. If you run

```
make non_blocking_queue.tested
```

two very simple tests will run, and the second **will fail**. You should modify the code in `non_blocking_queue.tests.c` to contain suitable tests of the correctness of your non-blocking queue, and remove the two example tests. If you want an example of how to write some more realistic tests, looking in `list.tests.c` may help. We are using assertions to check properties during the tests. If you need details about assertions, these can be found in Appendix A.

4.3 Implementing a bounded queue

The aim of this section is to implement a bounded queue of `unsigned int` values which may be useful in later parts. A **bounded queue** is a fixed size queue where pop calls block if no data is available, until another thread pushes a value, and push calls block if there is no free space to store a new value until a value is popped.

As with the non-blocking queue, even if you don't require this component later, **you should still implement it** as the code will be assessed. You should avoid using the provided linked list implementation for this components, as it has different memory management requirements.

To get you started, an outline implementation is provided in `bounded_queue.h` and `bounded_queue.c`. The header file has the following contents:

```
#ifndef _BOUNDED_QUEUE_H_
#define _BOUNDED_QUEUE_H_

#include <pthread.h>
#include <semaphore.h>

typedef struct BoundedQueue {
  /* Add fields as needed */
  int remove_me; // Added to silence empty structure compiler warning
} BoundedQueueT;

void bounded_queue_create(BoundedQueueT* queue, unsigned int capacity);
void bounded_queue_destroy(BoundedQueueT* queue);

int bounded_queue_push(BoundedQueueT* queue, unsigned int value);
int bounded_queue_pop(BoundedQueueT* queue, unsigned int* value);
```

```
int bounded_queue_empty(BoundedQueueT* queue);
unsigned int bounded_queue_length(BoundedQueueT* queue);

void bounded_queue_terminate(BoundedQueueT* queue);


#endif
```

You should modify the code in `bounded_queue.c` as follows:

- Add code so `bounded_queue_create` and `bounded_queue_destroy` create and destroy a queue of the specified capacity, allocating and releasing resources as required. To do so, you may need to add fields to the `BoundedQueue` structure in `bounded_queue.h`.
- Add code so that `bounded_queue_empty` returns non-zero if and only if the queue is empty.
- Add code so that `bounded_queue_length` returns the length of the queue.
- Add code to `bounded_queue_push`, `bounded_queue_pop` and `bounded_queue_terminate` so that:
    1. if `bounded_queue_terminate` is called, then all subsequent calls to `bounded_queue_push` and `bounded_queue_pop` fail and return non-zero.
    2. if `bounded_queue_terminate` hasn't been called, and the queue has available space, then the input value of `bounded_queue_push` is added to the back of the queue.
    3. if `bounded_queue_terminate` hasn't been called, and the queue is non-empty, then when `bounded_queue_pop` is called, front of the queue is removed and placed in `*value`, and zero is returned.
    4. if `bounded_queue_terminate` hasn't been called, and the queue is empty then `bounded_queue_pop` blocks. If `bounded_queue_terminate` is called, it fails and returns non-zero. Otherwise, if a value is subsequently pushed, it behaves as in case 3.
    5. if `bounded_queue_terminate` hasn't been called, and the queue is full then `bounded_queue_push` blocks. If `bounded_queue_terminate` is called, it fails and returns non-zero. Otherwise, if a value is subsequently popped, it behaves as in case 2.

The key idea is that pushing and popping using this queue **blocks** if no space or data is available. Blocking makes disposing of such a queue more complicated as we have to clean up potential blocked threads. To do this, terminating the queue makes blocked subsequent push and pop calls fail immediately.

You may also need to add other supporting code in these files, such as suitable type definitions or additional functions. You can also add or use functions from the files `utilities.h` and `utilities.c`.

The queue should be designed so it is safe to use in multi-threaded code.

You also should write some tests for your code. If you run

```
make bounded_queue.tested
```

two very simple tests will run, and the second **will fail**. You should modify the code in `bounded_queue.tests.c` to contain suitable tests of the correctness of your blocking queue, and remove the two example tests. If you want an example of how to write some more realistic tests, again, looking in `list.tests.c` may help. Details about assertions can be found in Appendix A.

4.4 Implementing priority queues

The aim of this section is to implement a priority queue.

To get you started, an outline implementation is provided in `priority_queue.h` and `priority_queue.c`. The header file has the following contents:

```
#ifndef _NON_BLOCKING_QUEUE_SET_H_
#define _NON_BLOCKING_QUEUE_SET_H_

#include "list.h"

typedef struct PriorityQueue {
  /* Add fields as needed */
  int remove_me; // Added to silence empty structure compiler warning
} PriorityQueueT;

typedef unsigned int PriorityT;

/* Standard interface, do not change*/
void priority_queue_create(PriorityQueueT* priority_queue, unsigned int number_of_priorities);
void priority_queue_destroy(PriorityQueueT* queue);

void priority_queue_push(PriorityQueueT* priority_queue, PriorityT level, unsigned int value);
int priority_queue_pop(PriorityQueueT* queue, unsigned int* value);

int priority_queue_empty(PriorityQueueT* queue);
unsigned int priority_queue_length(PriorityQueueT* queue);

void priority_queue_terminate(PriorityQueueT* priority_queue);
/* End of standard interface */

/* You may add functions to the queue interface if required */

#endif
```

You should modify the code in `priority_queue.c` for your implementation utilising a similar pattern to the bounded and non-blocking queues above. The

priority queue should be designed so it is safe and efficient to use in multi-threaded code.

You should modify the code in `priority_queue.tests.c` to contain suitable tests of the correctness of your priority queue. You may also need to add other supporting code in these files, such as suitable type definitions or additional functions. You can also add or use functions from the files `utilities.h` and `utilities.c`.

4.5 Simulating multiple CPUs

Now we have some supporting code, we start work on the main body of the simulator. The aim of this part is to create some threads in the simulator, which will simulate the available CPUs.

Add the following features in `simulator.c`:

- Modify the function `simulator_start` so that it starts `thread_count` threads, each running a function called `simulator_routine`. Each thread should be passed a unique identifying number.
- In `simulator_routine`, write out a message using the logger saying a thread with that identifying number has started. For now the thread can then simply finish.
- Modify the function `simulator_stop` so that it waits for all the threads to finish, and does any other necessary clean up.

You can briefly check your code by recompiling `coursework` and running it. It should now print out the additional logger messages that you added, and then terminate.

4.6 Managing process ids

As we are simulating an operating system, we will need to manage process identities. A process id should be stored in the `ProcessIdT` type, which is simply an `unsigned int`.

Modify `simulator.c` to efficiently store process ids in such a way that:

- The argument `max_processes` of `simulator_start` specifies the maximum number of process ids available.

- Process ids can be requested when needed. Such requests should block if no ids are available.
- Process ids can be returned when no longer needed.

You can briefly check your code by recompiling `coursework` and running it. You shouldn't see much change, but the code should still run and complete successfully.

By efficiently, we mean your implementation should have time complexity for issuing and returning process ids which is better than linear in the maximum number of processes.

4.7 Simulating running processes and waiting for them to finish

The aim of this part is to add support in the simulator for creating processes and waiting for them to finish. This is a challenging step, as several changes are needed to achieve the desired functionality.

Your operating system design should provide one ready queue per CPU, rather than a single global queue for access to all the CPUs. For now this will mean once a process is created, it will always run on the same simulated CPU. Each new process should be added to the next CPU in a round robin manner. For example, on a 2 CPU system:

- The first process created should go in the ready queue for CPU 0.
- The second process created should go in the ready queue for CPU 1.
- The third process created should go in the ready queue for CPU 0.
- The fourth process created should go in the ready queue for CPU 1.
- And so on ...

Modify `simulator.c` as follows:

- Add code so that `simulator_create_process` creates a new process, stores the necessary information to manage that process in a process table, and enters its process id into a suitable ready queue. Note this is a simplification which creates a process to run specified code in one step, rather than using the fork and exec pattern discussed in lectures. You should also add a log message indicating which process id has been created.

- Add code so that `simulator_routine` waits for processes in the ready queue for that CPU, and removes and runs them using the evaluator. Details of how to use the evaluator code are given in Appendix B. For now you can assume processes will either terminate or have their timeslice end, and your code should handle these cases appropriately.
- Add code so that `simulator_wait` waits for a specified process id to finish, blocking if necessary, and then cleans up any data associated with that process. The process id should be recycled so it can be used again. You should also add a log message indicating which process id is being waited for when the function is called.
- Add code to `simulator_start` to set up any needed resources.
- Add code to `simulator_stop` to shut down the simulator and clean up resources.

4.8 Making the environment create and wait for processes

Now we are able to create and wait for the completion of processes in the simulator, the next step is to extend the environment so that it exercises this functionality.

To do so, you should modify `environment.c` as follows:

- Add code so that `environment_start` creates `thread_count` threads, each running a function called `terminating_routine` (that simulates CPU bound processes without IO).
- Add code so that `environment_stop` waits for these threads to finish, and then cleans up any resources you have used.
- Add code so that `terminating_routine` loops `iterations` times. In each loop it:
  - Calls `simulator_create_process` `batch_size` times, passing code that terminates after a variable number of steps between 1 and 20, using the provided function `evaluator_terminates_after`.
  - Waits for each of these processes to finish, using `simulator_wait`.

If you recompile and run the coursework, you should see log messages showing the new features in action. Test your code under various conditions to ensure that it functions in all cases (e.g., consider the case where the number of processes "wanting to enter the system" at any one time is larger than the number of available process IDs.

4.9 Dealing with blocking processes

The aim of this part is to add IO events by modifying the event source to generate events, and modifying the simulator to handle then.

Modify `event_source.c` as follows:

- Add code so that `event_source_start` creates a thread which calls `simulator_event` every `interval` microseconds. You may use the standard function `usleep` to wait for the required time.
- Add code so that `event_source_stop` terminates the event source, waits for it to finish, and cleans up any resources.

Modify `simulator.c` as follows:

- Add code so that `simulator_routine` adds blocked processes to a suitable event queue. We make the simplifying assumption that all blocked processes are waiting on the same simple event source.
- Add code so that `simulator_event` immediately moves the front of the event queue to the ready queue when a new event arrives. You should also write a log message indicating which process id has been moved to the ready queue.
- Add code to `simulator_start` to set up any needed resources.
- Add code to `simulator_stop` to shut down the simulator and clean up resources.

4.10 Making the environment create and wait for jobs that may block

Now we have added support for IO events, the next step is to extend the environment so that it exercises this functionality, by creating processes that may block. This will be done by adding additional threads to the environment, following a very similar pattern to Section 4.8.

To do so, you should modify `environment.c` as follows:

- Add code so that `environment_start` creates `thread_count` threads, each running a function called `blocking_routine`.

- Add code so that `environment_stop` waits for these threads to finish, and then cleans up any resources you have used.
- Add code so that `blocking_routine` loops `iterations` times. In each loop it:
  - Calls `simulator_create_process` `batch_size` times, passing code that terminates after 5 steps, intermittently blocking on IO, using the provided function `evaluator_blocking_terminates_after`.
  - Waits for each of these processes to finish, using `simulator_wait`.

If you recompile and run the coursework, you should see log messages showing the new features in action.

4.11 Priority Boosting

We shall implement a priority booster that runs at regular intervals. The booster triggers an increment of the level of a lower priority process on each simulated CPU at regular intervals. A process may end up being boosted more than once. Once the process has run, it should return to its initial priority level.

A trivial booster daemon is provided in the files `booster_daemon.c` and `booster_daemon.h`. Modify these files so that:

- `booster_daemon_start` runs a thread boosting the priority of processes at intervals specified by the `interval` argument. Each interval, the daemon should signal to the simulator that it should boost a process priority. You will need to make simulator modifications to do the actual boosting. The criteria by which a process is chosen to be boosted is left up to you, but a process should be boosted on each simulated CPU.
- `booster_daemon_stop` stops the daemon thread, and cleans up any resources.

4.12 CPU Load Rebalancing

Over time, it is likely that the number of processes waiting for each of our simulated CPUs becomes unbalanced. In this final part we will implement a mechanism to periodically rebalance the number of processes assigned to each CPU.

You should modify `simulator.c` so that:

- The parameter `rebalance_interval` of `simulator_start` specifies the number of `simulator_create_process` calls that should trigger rebalancing. For example if `rebalance_interval` is 100, then on every 100th call to `simulator_create_process`, the simulator should rebalance the number of processes assigned to each CPU to be as uniform as possible.
- Rebalancing should change which CPU queue a processes joins *after* it is next run.

5 Submitting your work

Please submit your project using the course Moodle submission page, before the hard deadline specified in Moodle. It is highly recommended that you submit your work before the soft deadline specified in Moodle, as if you encounter problems it will be easier to get support, and there will be sufficient time to correct any issues. You can submit work via Moodle multiple times, and it is a good idea to do so to avoid last minute problems. Only the final version submitted will be marked.

In order to submit your work, complete the following steps:

1. Make a zip file of your latest course code, by typing:

   ```
   make coursework.tar.gz
   ```

2. Copy the generated `coursework.tar.gz` file to a fresh directory, and extract it again, using the command:

   ```
   tar zxvf coursework.tar.gz
   ```

   The files in this directory should be your latest work. Do a quick double check that this is the case.

3. Use the Moodle submission page for COMP2007 to submit your generated `coursework.tar.gz` file.

Please follow the instructions above to submit your work. Submitting your work in a different format may impact your mark.

6 Where to find help

If you have problems with the coursework instructions, or technical issues such as with the Linux environment either:

- Ask during one of the lab sessions (preferred option).
- Ask after lectures.
- Schedule an office hours meeting.

If you are having trouble solving the programming tasks, we cannot provide direct help with that as it is part of the assessment. You could consider:

- Re-reading the material from the processes and concurrency lectures, and possibly watching the lecture recordings.
- Checking if anything covered in the lab work can help.
- Referring to the standard Posix Threads documentation online.

7 Assessment guidelines

7.1 General guidelines

- Marks will be awarded for completing the various stages of the coursework.
- Good quality code will receive higher marks. Specifically, the following will cost marks:
  - Concurrency bugs such as race conditions or deadlocks.
  - Poor concurrent programming, such as locks with unnecessarily large scopes that hinder threads making progress.
  - Leaking memory or other resources.
  - Code that cannot be shut down cleanly.
- You are expected to use the concurrency features discussed in lectures, specifically Posix threads, mutexes and semaphores. Please don't use things we haven't discussed, such as Posix condition variables.
- Good software engineering will be rewarded, particularly practices that make your code clear and easy to understand, such as:

- Clear naming choices, e.g. `process_id` rather than `x`.
- Consistency in design decisions - avoid doing similar things in arbitrarily different ways, or using different naming conventions in different parts of the code.
- Avoiding dead or redundant code.
- Eliminating unnecessary duplication of similar code - avoid cut and paste programming.
- Not adding features that were not required in the specification.
- Adding comments or assertions as appropriate to clarify your code.

7.2 AI Usage

This coursework assesses your knowledge of concurrency and operating systems, and how you can apply that knowledge in practice. **The use of AI tools is not permitted for this coursework**. This includes:

- Chat tools such as ChatGPT
- Agentic software engineering tools such as Claude Code.
- In editor AI support such as GitHub Copilot.

Use of such tools will be considered academic misconduct.

8 Academic Misconduct

This is an individual project. Academic misconduct will be followed up on. Please do **not**

- Discuss your work with others.
- Share your work with other students.
- Copy work from other students.
- Submit work that is not your own.

In particular, instead of doing any of the above:

- If you cannot understand part of the coursework, ask the lecturers or TAs in labs for assistance. This will also allow the lecturers to pass on clarifications to other students.
- If you have technical difficulties, such as with connecting to the Linux environment, or building the provided code, again please ask in labs. In order for this to be possible, **please at least start on the coursework early enough to iron out technical issues**. It may not be possible to address technical problems during the holidays or very close to the hand-in deadline.
- If you encounter genuine exceptional circumstances, please use the standard university EC mechanism to address this. Asking for assistance from your fellow students under such circumstances is not appropriate, and may also drag them into misconduct issues.

Appendices

A Background on assertions

The coursework uses a simple feature provided by the C standard library called **assertions**. As these aren't covered in first year courses, we will briefly introduce the idea.

Assertions are used to check conditions that you expect to hold in your code, typically in a debug build of your program. The makefile provided with the coursework is already configured to produce debug builds of your code, so you don't need to worry about how to set this up.

Consider the following artificial example.

```
#include <assert.h>

int main() {
  assert(1 + 1 == 2);
  assert(2 - 3 == 1);
}
```

The inclusion `#include <assert.h>` includes the definitions needed to use assertions. There are two assertions in the code:

1. The first `assert(1 + 1 == 2);` checks a condition that is obviously true. Nothing happens, and the code continues running.

2. The second `assert(2 - 3 == 1);` checks a condition that is obviously false. The program will deliberately crash at this point, and report a failed assertion.

Forcing your code to crash seems odd at first, but it is very useful in practice as:

- You can check for your program being in "impossible" states, forcing bugs to reveal themselves earlier.
- If an assertion does force your program to fail, you can load the program up in a debugger, and analyze what is going wrong.
- You can use assertions to document expectations. For example the code:

```
int my_function(void* pointer) {
  assert(pointer);
  // The rest of the code
  ...
}
```

  makes very clear to callers of `my_function` that null pointers should not be passed as arguments to this function.

- You can use assertions in unit tests to verify your code is correct. For example, the code:

```
void test_empty_creation_destruction() {
  printf("Test empty creation/destruction\n");
  ListT* list = list_create();
  assert(list_empty(list));
  assert(list_length(list) == 0);
  list_destroy(list);
}
```

  checks a linked list implementation to verify that newly created lists are empty, and therefore have zero length. More examples of using assertions in tests can be found in the provided unit test file `list.tests.c`.

If needed, more background on assertions can be found in any standard source on C programming, such as the Kernighan and Ritchie book.

B Using the evaluator

The evaluator is intended to simplistically simulate running some code. Examining the header file, the key elements are:

```
typedef enum Reason {
  reason_terminated,
  reason_timeslice_ended,
  reason_blocked,
} ReasonT;

typedef struct EvaluatorResult {
  unsigned int PC;
  unsigned int cpu_time;
  ReasonT reason;
} EvaluatorResultT;

typedef struct EvaluatorCode {
  EvaluatorResultT (*implementation)(unsigned int, unsigned int);
  unsigned int parameter;
} EvaluatorCodeT;


// The evaluator - pretends to run some code on a CPU
EvaluatorResultT evaluator_evaluate(EvaluatorCodeT const code, unsigned int PC);
```

You wont need to understand the details of the type `EvaluatorCodeT`, as they are implementation detail of the evaluator.

The function `evaluator_evaluate` pretends to run the specified code, with the initial program counter set to `PC`. The PC should be set to zero on the first run of any code. The return value is a structure containing:

- The new program counter value. This should be used when running code a subsequent time, so it restarts from the right place.
- A reason why evaluation stopped. This can either be because a timeslice ended, the process terminated, or the process blocked.
- The amount of hypothetical CPU time used during evaluation. The time taken in the evaluator will be proportional to this hypothetical value to give some plausible variation in timings.

Examples:

- To run a CPU bound process that terminates after five runs, with initial PC of zero, call

  ```
  evaluator_evaluate(evaluator_terminates_after(5), 0);
  ```

- To run a CPU bound process that runs indefinitely, with initial PC of zero, call

  ```
  evaluator_evaluate(evaluator_infinite_loop, 0);
  ```

- To run a process that may occasionally block, and terminates after five runs, with initial PC of zero, call

```
evaluator_evaluate(evaluator_blocking_terminates_after(5),
  0);
```

More example usage can be found in `evaluator.tests.c`.